

SQL INTERVIEW QUESTIONS

Q1. What is SQL?

Ans.

SQL (Structured Query Language) is a standard language used to communicate with databases.

Key Points:

- It is used to **store, retrieve, update, and delete data** in a database.
- SQL works with **Relational Database Management Systems (RDBMS)** like MySQL, Oracle, SQL Server.
- It is a **declarative language**, meaning you tell *what you want*, not *how to do it*.

Main Uses:

- **Query data** → using **SELECT**
- **Insert data** → using **INSERT**
- **Update data** → using **UPDATE**
- **Delete data** → using **DELETE**

Example:

```
SELECT name FROM employees;
```

This query fetches employee names from the table.

Important Note:

- SQL is not a database itself, it is just a **language to interact with databases**.
-

Q2. What is DBMS?

Ans.

DBMS (Database Management System) is software used to **store, manage, and organize data** in a structured way.

Key Points:

- It acts as an **interface between users and the database**.
- Helps in **creating, reading, updating, and deleting data (CRUD operations)**.
- Ensures **data security, consistency, and integrity**.
- Supports **multi-user access** without data conflicts.

Main Functions:

- **Data Storage** → Stores data in tables/files
- **Data Retrieval** → Fetch data efficiently
- **Data Security** → Controls access using permissions
- **Data Backup & Recovery** → Prevents data loss

Examples:

- MySQL, Oracle, SQL Server, PostgreSQL

Important Note:

- DBMS is the **software**, while the database is the **actual data stored inside it**.

Q3. What is the difference between SQL and DBMS ?

Ans.

1. Definition

- **SQL (Structured Query Language)** → A language used to interact with databases
- **DBMS (Database Management System)** → A software/system used to store and manage data

2. Purpose

- SQL → Used to **query and manipulate data**
- DBMS → Used to **create, manage, and control databases**

3. Nature

- SQL → Declarative programming language (you say *what you want*)
- DBMS → Application software that handles data operations

4. Usage

- SQL → Executes commands like **SELECT, INSERT, UPDATE, DELETE**
- DBMS → Provides environment to run SQL queries and manage data

5. Dependency

- SQL → Cannot work alone without a DBMS
- DBMS → Can exist without SQL (some use other query languages)

SQL → Queries written by developers

DBMS → MySQL, Oracle, SQL Server

Quick Summary:

- SQL is a **tool (language)**
- DBMS is the **platform (software)** that uses that tool

Q4. What is RDBMS?

Ans. RDBMS (Relational Database Management System) is a type of DBMS that **stores data in the form of tables (relations)**.

Key Points:

- Data is organized into **rows (records)** and **columns (fields)**.
- Each table is related to other tables using **keys**.
- Follows the concept of **relations between data**.

Important Concepts:

- **Primary Key** → Uniquely identifies each record
- **Foreign Key** → Creates relationship between tables
- **Tables** → Structured format to store data

Features:

- Ensures **data integrity and consistency**
- Supports **normalization** to reduce redundancy
- Allows **multiple users** to access data safely

Example:

```
SELECT * FROM students;
```

This retrieves all data from a table in an RDBMS.

Examples of RDBMS:

- MySQL, Oracle, SQL Server, PostgreSQL

All RDBMS are DBMS, but **not all DBMS are RDBMS** (because RDBMS specifically uses tables and relationships).

Q5. Difference between DBMS and RDBMS?

Ans.

1. Definition

- **DBMS (Database Management System)** → Software to **store and manage data**
- **RDBMS (Relational DBMS)** → Advanced DBMS that stores data in **tables with relationships**

2. Data Storage

- **DBMS** → Data stored in **files or simple structures**
- **RDBMS** → Data stored in **tables (rows & columns)**

3. Relationships

- **DBMS** → **No proper relationships** between data
- **RDBMS** → Uses **Primary Key & Foreign Key** to create relationships

4. Normalization

- **DBMS** → **Not strictly followed**
- **RDBMS** → **Follows normalization** to reduce redundancy

5. Data Integrity

- **DBMS** → **Low data integrity**
- **RDBMS** → **High data integrity** with constraints

6. Security & Multi-user Access

- **DBMS** → **Limited security, single-user or small systems**
- **RDBMS** → **Strong security, supports multiple users simultaneously**

7. Examples

- **DBMS** → File System, XML-based storage
- **RDBMS** → MySQL, Oracle, SQL Server

Quick Summary:

- **DBMS** = **basic data storage system**
- **RDBMS** = **structured, relational, and more powerful DBMS**

Q6. What is NoSQL?

Ans. NoSQL (Not Only SQL) is a type of database designed to **store and manage unstructured or semi-structured data**.

Key Points:

- It does **not use traditional tables (rows & columns)** like RDBMS
- Uses flexible formats like **JSON, key-value, document, or graph**
- Designed for **high scalability and large data (Big Data)**
- Schema is **dynamic (no fixed structure)**

Types of NoSQL Databases:

- **Key-Value Stores** → Redis
- **Document Databases** → MongoDB
- **Column-Based** → Cassandra
- **Graph Databases** → Neo4j

Features:

- **High performance** for large-scale applications
- **Easy to scale horizontally (add more servers)**
- **Handles real-time data and big data** efficiently

Example (MongoDB style):

```
{ name: "Chandu", age: 22 }
```

NoSQL is used when data is **large, flexible, and rapidly changing**, unlike RDBMS which is structured and fixed.

Q7. What are tables and schemas?

Ans. Tables are the basic structure in a database where data is stored in the form of **rows and columns**. Each table represents a specific entity like employees, students, or orders. A row represents a single record, and a column represents a field or attribute. For example, an employees table can have columns like id, name, and salary. Tables help in organizing data in a structured and easy-to-query format.

A schema is the **logical structure or blueprint of the database**. It defines how data is organized, including tables, columns, data types, and relationships between tables. It acts like a design that shows how the database is built.

Tables store the actual data, while the schema defines **how that data is structured**. A database can have multiple schemas, and each schema can contain multiple tables.

```
CREATE TABLE employees (  
  id INT,  
  name VARCHAR(50),  
  salary INT  
);
```

Here, the table is "employees" and its structure (columns and types) is part of the schema.

Q8. What is a primary key?

Ans. A primary key is a column (or set of columns) used to **uniquely identify each record** in a table. It ensures that no two rows have the same value for that column. It is one of the most important constraints in a relational database.

A primary key must have **unique values** and it **cannot contain NULL values**. This guarantees that every record can be identified clearly. Each table can have only **one primary key**, but it can be made of multiple columns (called a composite key).

Primary keys also help in creating relationships with other tables using foreign keys. They improve **data integrity and faster searching**.

```
CREATE TABLE students (  
  id INT PRIMARY KEY,  
  name VARCHAR(50)  
);
```

Here, "id" is the primary key, so every student must have a unique id.

Q9. What is a foreign key?

Ans. A foreign key is a column (or set of columns) used to **create a relationship between two tables**. It links one table to another by referring to the **primary key** of the parent table.

It ensures **referential integrity**, meaning you cannot insert a value that does not exist in the referenced table. This helps maintain correct and consistent data across tables.

A foreign key can have **duplicate values** and it can also contain **NULL values** (unless restricted). Multiple foreign keys can exist in a single table.

It is mainly used to represent relationships like **one-to-many** (for example, one department has many employees).

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  FOREIGN KEY (customer_id) REFERENCES customers(id)  
);
```

Here, "customer_id" is a foreign key that refers to "id" in the customers table.

Q10. What is a unique key?

Ans. A unique key is a constraint used to ensure that **all values in a column are unique**, meaning no duplicate values are allowed. It helps maintain **data uniqueness** similar to a primary key.

The main difference is that a unique key can **accept NULL values** (in most databases, one or more NULLs are allowed), while a primary key cannot have NULL.

A table can have **multiple unique keys**, but only one primary key. Unique keys are often used for fields like email, phone number, etc., where duplication should not happen.

It also helps in maintaining **data integrity** and can be used for indexing to improve search performance.

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  email VARCHAR(100) UNIQUE  
);
```

Here, "email" must be unique for every user, but it can be NULL depending on database rules.

Q11. What is a composite key?

Ans. A composite key is a primary key that is made up of **two or more columns combined together** to uniquely identify each record in a table. It is used when a single column is not enough to ensure uniqueness.

Each column alone may have duplicate values, but the **combination of those columns will always be unique**. This is common in tables that represent relationships between entities.

Composite keys help maintain **data integrity** and avoid duplicate records when multiple attributes define uniqueness.

For example, in a student-course table, one student can enroll in many courses, and one course can have many students. So we combine `student_id` and `course_id`.


```
CREATE TABLE enrollments (  
    student_id INT,  
    course_id INT,  
    PRIMARY KEY (student_id, course_id)  
);
```

Here, the combination of `student_id` and `course_id` forms a composite key, ensuring each enrollment is unique.

Q12. What is a candidate key?

Ans. A candidate key is a column (or set of columns) that can **uniquely identify each record** in a table. It is a potential key that satisfies all the conditions of a primary key.

A table can have **multiple candidate keys**, but only one of them is chosen as the **primary key**. The remaining candidate keys are called **alternate keys**.

A candidate key must have **unique values** and **cannot contain NULL values**. It should uniquely identify every row without any duplication.

For example, in a users table, both **email** and **phone number** can uniquely identify a user, so both are candidate keys.

```
CREATE TABLE users (  
    id INT,  
    email VARCHAR(100),  
    phone VARCHAR(15),  
    PRIMARY KEY (id),  
    UNIQUE (email),  
    UNIQUE (phone)  
);
```

Here, `id`, `email`, and `phone` are candidate keys, but `id` is selected as the primary key.

Q13. What are SQL constraints?

Ans. SQL constraints are rules applied on table columns to **restrict and control the type of data** that can be inserted into a database. They help maintain **data accuracy, integrity, and consistency**.

Constraints ensure that only valid data is stored and prevent invalid or duplicate entries. They can be applied while creating a table or added later.

Common types of constraints:

- **PRIMARY KEY** → Uniquely identifies each record and cannot be NULL
- **FOREIGN KEY** → Maintains relationship between tables
- **UNIQUE** → Ensures all values in a column are unique
- **NOT NULL** → Prevents NULL values in a column
- **CHECK** → Ensures a condition is satisfied before inserting data
- **DEFAULT** → Assigns a default value if no value is provided

These constraints improve **data reliability and database performance** by enforcing rules at the database level.

```
CREATE TABLE employees (  
  id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE  
);
```

Here, constraints ensure id is unique, name cannot be NULL, and email must be unique.

Q14. What is NOT NULL constraint?

Ans. The NOT NULL constraint is used to ensure that a column **cannot store NULL (empty) values**. It means every record must have a valid value for that column.

This constraint is important for fields that are **mandatory**, like name, email, or id, where missing data is not allowed.

It helps maintain **data completeness and integrity** by preventing incomplete records from being inserted into the table.

If you try to insert a NULL value into a NOT NULL column, the database will **throw an error**.

It can be applied while creating a table or added later using the ALTER command.

```
CREATE TABLE employees (  
  id INT,  
  name VARCHAR(50) NOT NULL  
);
```

Here, "name" must always have a value; it cannot be left empty.

Q15. What is the DEFAULT constraint?

Ans. The DEFAULT constraint is used to **assign a default value** to a column when no value is provided during insertion. It helps avoid NULL values and ensures consistent data.

If the user does not specify a value for that column, the database will **automatically insert the default value**.

It is useful for fields that should have a common value, like status, country, or created_date.

The DEFAULT value must match the **data type of the column**. It can be a constant value or sometimes a system value like current date.

It improves **data consistency and reduces manual effort** while inserting records.

```
CREATE TABLE employees (  
    id INT,  
    status VARCHAR(20) DEFAULT 'active'  
);
```

Here, if no status is provided, it will automatically be set to "active".

Q16. What is the CHECK constraint?

Ans. The CHECK constraint is used to **apply a condition on a column** so that only values that satisfy that condition can be inserted or updated.

It helps enforce **business rules at the database level**, ensuring only valid data is stored.

If a value does not meet the condition, the database will **reject the operation with an error**.

CHECK can be applied to a single column or multiple columns and can include logical conditions like >, <, =, BETWEEN, etc.

It improves **data accuracy and integrity** by restricting invalid entries.

```
CREATE TABLE employees (  
  id INT,  
  age INT CHECK (age >= 18)  
);
```

Here, only employees with age 18 or above can be inserted into the table.

Q17. What is a view?

Ans. A view is a **virtual table** in SQL that is created from the result of a query. It does not store data itself; instead, it displays data from one or more tables.

A view is created using a SELECT statement, and whenever you query the view, it fetches the latest data from the underlying tables.

It is mainly used to **simplify complex queries**, improve readability, and provide a specific representation of data.

Views also help in **security**, as you can restrict users to see only certain columns or rows instead of the full table.

There are two types: **simple view** (based on one table) and **complex view** (based on multiple tables).

```
CREATE VIEW employee_view AS  
SELECT name, salary FROM employees;
```

Here, employee_view shows only name and salary from the employees table.

Q18. What is an index?

Ans. An index is a database object used to **improve the speed of data retrieval** from a table. It works like an index in a book, helping the database find data quickly without scanning the entire table.

Indexes are created on one or more columns and store a **sorted reference of data**, which makes searching faster.

They are mainly used with queries that use **WHERE, JOIN, ORDER BY, GROUP BY** clauses.

There are different types like **single-column index**, **composite index**, **unique index**.

While indexes improve read performance, they can **slow down insert, update, and delete operations** because the index also needs to be updated.

So, indexes should be used carefully on columns that are frequently searched.

Simple example:

```
CREATE INDEX idx_name ON employees(name);
```

Here, an index is created on the name column to speed up search operations.

Q19. What are different types of indexes?

Ans. Different types of indexes are used to **optimize query performance** based on use cases.

- **Single Column Index** → Created on one column to speed up searches on that column
- **Composite Index** → Created on multiple columns; useful when queries filter on more than one column
- **Unique Index** → Ensures all values in the column are **unique** (no duplicates allowed)
- **Primary Index** → Automatically created when a **primary key** is defined
- **Clustered Index** → Determines the **physical order of data** in the table (only one per table)
- **Non-Clustered Index** → Stores a **separate structure** with pointers to actual data (multiple allowed)
- **Full-Text Index** → Used for **searching text data** like words or phrases in large text columns

Indexes improve **read/query performance**, but too many indexes can slow down **insert, update, delete operations**.

Simple example:

```
CREATE INDEX idx_emp ON employees(name, department);
```

This is a composite index on name and department columns.

Q20. What is a clustered index?

Ans. A clustered index is an index that **defines the physical order of data in a table**. It means the actual table data is stored in the same order as the index.

Because data itself is sorted, searching and range queries become **very fast**.

A table can have **only one clustered index**, since data can be arranged in only one order.

By default, when you create a **primary key**, most databases automatically create a clustered index on that column.

Clustered indexes are best used on columns that are **frequently searched or sorted**, like id or date.

It is especially efficient for **range queries** (for example, fetching records between two values).

Simple example:

```
CREATE CLUSTERED INDEX idx_id ON employees(id);
```

Here, the table data will be physically ordered based on the id column.

Q21. What is a non-clustered index?

Ans. A non-clustered index is an index that **stores a separate structure from the actual table data**. It contains the indexed column values along with a **pointer (reference)** to the actual rows.

Unlike a clustered index, it **does not change the physical order of data** in the table.

A table can have **multiple non-clustered indexes**, which helps optimize different types of queries.

It is useful for columns that are **frequently used in search conditions** but are not part of the primary key.

When a query runs, the database first searches the index, then uses the pointer to fetch the actual data.

Non-clustered indexes improve **read performance**, but like all indexes, they add some overhead to **insert, update, delete operations**.

Simple example:

```
CREATE INDEX idx_name ON employees(name);
```

Here, a non-clustered index is created on the name column to speed up search queries.

Q22. What is normalization?

Ans. Normalization is the process of **organizing data in a database** to reduce redundancy and improve data integrity. It involves dividing large tables into smaller related tables and defining relationships between them.

The main goal is to **eliminate duplicate data** and ensure data is stored logically. This helps avoid problems like data inconsistency and update anomalies.

Normalization is done using a set of rules called **normal forms**. The most common are 1NF, 2NF, and 3NF.

- **1NF (First Normal Form)** → Removes repeating groups, ensures each column has atomic values
- **2NF (Second Normal Form)** → Removes partial dependency (depends on full primary key)
- **3NF (Third Normal Form)** → Removes transitive dependency (no indirect dependency)

Normalization improves **data consistency, reduces storage usage, and makes maintenance easier**.

But too much normalization can sometimes affect performance, so in real projects, a balance is maintained.

Q23. What is denormalization?

Ans. Denormalization is the process of **combining tables or adding redundant data** to improve query performance. It is the opposite of normalization.

In denormalization, we intentionally **introduce redundancy** to reduce the number of joins required while fetching data. This makes data retrieval **faster**, especially in large and complex queries.

It is mainly used in **read-heavy applications** like reporting systems and data warehouses.

For example, instead of joining orders and customers tables, we may store **customer_name directly in the orders table**.

Advantages include **faster query performance and simpler queries**.

Disadvantages include **data redundancy, increased storage, and risk of inconsistency** during updates.

So, denormalization is used when **performance is more important than strict data integrity**, and it is usually applied carefully

Q24. What are different types of SQL commands?

Ans. Different types of SQL commands are used to perform specific operations in a database. They are mainly classified into the following categories:

- **DDL (Data Definition Language)** → Used to define and modify database structure
Commands: **CREATE, ALTER, DROP, TRUNCATE**
Example: creating tables, modifying columns
- **DML (Data Manipulation Language)** → Used to manage data inside tables
Commands: **INSERT, UPDATE, DELETE**
Example: adding or updating records
- **DQL (Data Query Language)** → Used to retrieve data
Command: **SELECT**
Example: fetching data from tables
- **DCL (Data Control Language)** → Used for access control and permissions
Commands: **GRANT, REVOKE**
Example: giving user access rights
- **TCL (Transaction Control Language)** → Used to manage transactions
Commands: **COMMIT, ROLLBACK, SAVEPOINT**
Example: saving or undoing changes

These categories help in **organizing SQL operations clearly** and are important for database management and control.

Q25. What is the SELECT statement?

Ans. The SELECT statement is used to **retrieve data from one or more tables** in a database. It is the most commonly used SQL command.

It allows you to **choose specific columns or all columns** from a table using *****.

You can filter data using conditions with **WHERE clause**, sort results using **ORDER BY**, and group data using **GROUP BY**.

It is part of **DQL (Data Query Language)** and does not modify data, only fetches it.

You can also use it with joins to **combine data from multiple tables**.

Simple example:

```
SELECT name, salary FROM employees;
```

This query retrieves name and salary columns from the employees table.

Using SELECT effectively is very important for **data analysis and reporting** in real-world applications.

Q26. What is the WHERE clause?

Ans. The WHERE clause is used to **filter records** from a table based on a specific condition. It allows you to retrieve only the data that matches your criteria.

It is commonly used with commands like **SELECT, UPDATE, DELETE** to apply conditions on rows.

The condition can use operators like **=, >, <, >=, <=, !=**, and also logical operators like **AND, OR, NOT**.

It helps in reducing the result set by selecting only **relevant data** instead of the entire table.

You can also use it with functions like **LIKE, IN, BETWEEN, IS NULL** for advanced filtering. Using WHERE properly is very important for writing **efficient and accurate queries**.

Q27. Difference between WHERE and HAVING?

Ans. Difference between WHERE and HAVING is mainly based on **when and how they filter data**:

- **WHERE clause** → Used to **filter rows before grouping**
Works on individual rows in a table
Cannot use aggregate functions like **SUM, COUNT, AVG**
Used with **SELECT, UPDATE, DELETE**
- **HAVING clause** → Used to **filter data after grouping**
Works on grouped results created by **GROUP BY**
Can use aggregate functions like **SUM, COUNT, AVG**
- **Execution Order**
WHERE → filters first
GROUP BY → groups data
HAVING → filters grouped data

Example for clarity:

```
SELECT department, COUNT(*)  
FROM employees  
WHERE salary > 50000  
GROUP BY department  
HAVING COUNT(*) > 5;
```

Here, WHERE filters rows first, and HAVING filters groups after aggregation.

Simple summary:

WHERE → filters rows

HAVING → filters grouped/aggregated data

Q28. What is GROUP BY?

Ans. The GROUP BY clause is used to **group rows that have the same values in specified columns**. It is mainly used with **aggregate functions** like **COUNT, SUM, AVG, MAX, MIN**.

It helps in performing calculations on **each group instead of the entire table**. For example, calculating total salary per department.

When GROUP BY is used, all selected columns must either be **in the GROUP BY clause or used with aggregate functions**. It is often combined with **HAVING** to filter grouped results. GROUP BY is very useful for **reporting and data analysis**. This query groups employees by department and returns the count of employees in each department.

Q29. What is ORDER BY?

Ans. The ORDER BY clause is used to **sort the result set** of a query based on one or more columns.

By default, it sorts data in **ascending order (ASC)**, but you can also sort in **descending order (DESC)**.

It helps in organizing the output in a **readable and meaningful way**, especially for reports.

You can sort by **single column or multiple columns**, and sorting happens from left to right priority.

ORDER BY is usually used with the **SELECT statement** after filtering or grouping.

This query displays employees sorted by salary in descending order (highest to lowest).

Using ORDER BY improves **data presentation and analysis**.

Q30. What is DISTINCT?

Ans. The DISTINCT keyword is used to **remove duplicate values** from the result set and return only **unique records**.

It is mainly used with the **SELECT statement** when you want to avoid repeated data.

If a column has duplicate values, DISTINCT will show **only one occurrence** of each value.

It can also be used on **multiple columns**, in which case the combination of values must be unique.

Using DISTINCT may slightly impact performance on large datasets, so it should be used when needed.

This query returns unique department names without duplicates.

DISTINCT is useful for **data cleaning and analysis** when identifying unique values.

Q31. What is LIMIT and OFFSET ?

Ans. LIMIT and OFFSET are used to **control how many rows you get and from where to start** in the result set. They are mainly used for **pagination (showing data page by page)**.

- **LIMIT** → Defines **how many rows to return**
- **OFFSET** → Defines **from which position to start returning rows**

Important point:

OFFSET starts from 0, meaning the first row is position 0, second is 1, third is 2, and so on.

Simple understanding:

OFFSET skips rows, LIMIT picks rows.

Example:

```
SELECT * FROM employees
```

```
LIMIT 5 OFFSET 0;
```

This returns the first 5 rows (starting from index 0).

```
SELECT * FROM employees
```

```
LIMIT 5 OFFSET 5;
```

This skips the first 5 rows (0–4) and returns next 5 rows (5–9).

So,

LIMIT = how many rows

OFFSET = where to start (starting index is always 0)

Q32. What is LIKE operator?

Ans. The LIKE operator is used to **search for a pattern in a column**. It is mainly used in the WHERE clause for **pattern matching**. It works with wildcard characters to match partial text instead of exact values.

Common wildcards:

- **%** → Represents **any number of characters** (including zero)
- **_** → Represents **exactly one character**

It is useful for searching names, emails, or any text-based data when you don't know the exact value.

LIKE is very useful for **flexible searching and filtering text data**.

Q33. What is the IN operator?

Ans. The IN operator is used to **check if a column value matches any value in a given list**. It is mainly used in the WHERE clause to filter multiple values in a simple way.

Instead of writing multiple OR conditions, IN makes the query **clean and readable**.

For example, instead of:

```
department = 'HR' OR department = 'IT' OR department = 'Sales'
```

you can use IN.

Simple example:

```
SELECT * FROM employees  
WHERE department IN ('HR', 'IT', 'Sales');
```

This returns employees who belong to HR, IT, or Sales.

It can also be used with numbers or even with a **subquery** to match dynamic values.

IN checks for **any match in the list**, so if the value exists in the given set, the row will be returned.

Q34. What is the BETWEEN operator?

Ans. The BETWEEN operator is used to **filter data within a specific range of values**. It is mainly used in the WHERE clause.

It works with numbers, dates, and even text, and it checks whether a value lies **between two given values (inclusive)**. That means it includes both the starting and ending values.

It helps make queries more readable instead of writing multiple conditions using AND.

Instead of writing:

```
salary >= 30000 AND salary <= 60000
```

you can use BETWEEN for simplicity.

Simple example:

```
SELECT * FROM employees  
WHERE salary BETWEEN 30000 AND 60000;
```

This returns employees whose salary is between 30000 and 60000, including both values.

It can also be used with dates:

```
SELECT * FROM orders  
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

BETWEEN is **inclusive**, meaning it includes both the lower and upper limits.

Q35. What is an alias in SQL?

Ans. Alias in SQL is a **temporary name given to a table or column** to make the query more readable and easy to write. It does not change the actual name in the database.

Aliases are mainly used when working with **long column names, multiple tables, or joins**, so the query becomes shorter and cleaner.

You can create an alias using the **AS** keyword, but it is **optional** in most databases.

For columns, aliases help in giving a **meaningful name to the output**.

For tables, alias helps in **shortening table names** and avoiding repetition.

Simple example:

```
SELECT name AS employee_name  
FROM employees;
```

Here, "employee_name" is an alias for the name column.

Alias exists only during query execution and is **not stored permanently** in the database.

Q36. What is the CASE statement?

Ans. The CASE statement in SQL is used to **apply conditional logic inside a query**, similar to an if-else statement in programming. It allows you to return different values based on conditions.

It is mainly used in the SELECT clause to **create custom output or derived columns** based on conditions.

There are two types:

- **Simple CASE** → compares a column with fixed values
- **Searched CASE** → uses conditions like >, <, =

It is very useful for **data transformation, categorization, and reporting**.

```
SELECT name,  
CASE  
  WHEN salary > 50000 THEN 'High'  
  WHEN salary BETWEEN 30000 AND 50000 THEN 'Medium'  
  ELSE 'Low'  
END AS salary_category  
FROM employees;
```

CASE executes conditions **in order** and returns the result of the first matching condition.

Q37. What is NULL?

Ans. NULL in SQL represents a **missing, unknown, or undefined value**. It is not the same as 0, empty string, or false — it simply means **no value is stored**.

A column can have NULL when data is **not available or not applicable**. For example, a middle_name column may be NULL if a person doesn't have one.

You cannot compare NULL using normal operators like = or !=. Instead, you must use **IS NULL** or **IS NOT NULL**.

Example:

```
SELECT * FROM employees  
WHERE manager_id IS NULL;
```

This returns employees who don't have a manager assigned.

NULL values can affect **calculations and aggregate functions**, as they are usually ignored in functions like COUNT, SUM, AVG (except COUNT(*)).

NULL means **absence of data**, not an actual value, so it must be handled carefully in queries.

Q38. Difference between NULL and 0?

Ans. Difference between NULL and 0 is based on meaning and behavior in SQL:

- **NULL** → Represents **no value / unknown / missing data**
It means data is not stored or not available
It is not equal to anything, even another NULL
- **0** → Represents an **actual numeric value**
It is a valid number and can be used in calculations normally
- **Comparison**
NULL cannot be compared using = or !=, you must use **IS NULL**
0 can be compared using normal operators like =, >, <
- **Calculations**
NULL is ignored in most aggregate functions like SUM, AVG
0 is included in calculations as a real value
- **Meaning in real-world example**
NULL → salary not entered yet
0 → salary is zero (valid value)

Simple summary:

NULL = **no data**

0 = **actual data value**

Q39. What is COUNT(), SUM(), AVG()?

Ans. COUNT(), SUM(), and AVG() are **aggregate functions** used to perform calculations on multiple rows and return a single result.

COUNT() → Used to **count the number of rows**

It can count all rows using **COUNT (*)** or only non-NULL values using a column

Example:

```
SELECT COUNT(*) FROM employees;
```

This returns total number of rows

SUM() → Used to **calculate the total of a numeric column**. It ignores NULL values

Example:

```
SELECT SUM(salary) FROM employees;
```

This returns total salary of all employees

AVG() → Used to **calculate the average of a numeric column**. It also ignores NULL values

Example:

```
SELECT AVG(salary) FROM employees;
```

This returns average salary

All these functions work on **multiple rows but return a single value**, so they are widely used in reporting and data analysis.

Q40. Difference between COUNT(*) and COUNT(column)?

Ans. Difference between COUNT(*) and COUNT(column) is based on **how they handle NULL values** and what they count:

- **COUNT(*)** → Counts **all rows in the table**
It includes rows even if they contain NULL values in any column
It simply counts the total number of records
- **COUNT(column)** → Counts **only non-NULL values** in that specific column
If a row has NULL in that column, it will **not be counted**

- **Behavior difference**

COUNT(*) → counts every row

COUNT(column) → skips rows where the column value is NULL

- **Example for clarity:**

```
SELECT COUNT(*), COUNT(salary) FROM employees;
```

If 10 rows exist but 2 salary values are NULL:

COUNT(*) = 10

COUNT(salary) = 8

Use COUNT(*) when you need **total rows**, and COUNT(column) when you need **only valid (non-NULL) data count**.

Q41. What is the ROUND() function?

Ans. The ROUND() function is used to **round a numeric value to a specified number of decimal places**. It helps in formatting numbers for better readability, especially in reports and calculations.

You can control how many digits you want after the decimal point by passing a second argument.

If the decimal part is **5 or more**, it **rounds up**, otherwise it rounds down.

If you don't specify the decimal places, it will **round to the nearest whole number**.

It is commonly used with aggregate functions like AVG or SUM to **limit decimal precision**.

Simple example:

```
SELECT ROUND(123.456, 2);
```

 —> Output → 123.46

Another example:

```
SELECT ROUND(123.456);
```

 —>Output → 123

ROUND() is useful for **controlling decimal precision and presenting clean output** in SQL queries.

Q42. What is the CONCAT() function?

Ans. The CONCAT() function is used to **join two or more strings into a single string**. It is commonly used to combine column values or text in a query result.

It takes multiple arguments and merges them **in the same order** as given. It works mainly with **text (string) data types**.

You can also add spaces or symbols between values to format the output properly.

It is useful for **creating full names, formatted messages, or combined fields** in reports.

CONCAT() returns a **single combined string**, and if any value is NULL (in some databases), the result may become NULL.

Q43.What is a JOIN?

Ans. A JOIN is used to **combine data from two or more tables** based on a related column, usually a **primary key and foreign key**. It allows you to retrieve related data stored in different tables as a single result.

In relational databases, data is stored in **separate tables to avoid redundancy**, so JOIN is required to bring that data together when needed.

For example, employee details may be in one table and department details in another. JOIN helps to **connect and fetch both together**.

Types of joins include **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN**, each deciding how matching and non-matching rows are handled.

JOIN works using a condition (ON clause) to **match rows between tables**.

Common types of joins:

- **INNER JOIN** → returns only matching records from both tables
- **LEFT JOIN** → returns all records from left table and matching from right
- **RIGHT JOIN** → returns all records from right table and matching from left
- **FULL JOIN** → returns all records from both tables

JOIN is very important because it enables **data normalization**, reduces duplication, and allows **efficient data retrieval across multiple related tables**.

Q44. Types of JOINS in SQL?

Ans. Types of JOINS in SQL are used to **combine data from multiple tables** in different ways based on matching conditions.

- **INNER JOIN** → Returns only the rows that have **matching values in both tables**
Non-matching rows are ignored
Most commonly used join
- **LEFT JOIN (LEFT OUTER JOIN)** → Returns **all rows from the left table** and matching rows from the right table
If no match, right table columns will be NULL
- **RIGHT JOIN (RIGHT OUTER JOIN)** → Returns **all rows from the right table** and matching rows from the left table
If no match, left table columns will be NULL
- **FULL JOIN (FULL OUTER JOIN)** → Returns **all rows from both tables**
If there is no match, NULL values are filled for missing sides
- **CROSS JOIN** → Returns **Cartesian product**, meaning every row from one table is combined with every row from another table

Choosing the right JOIN type is important for **correct data retrieval and performance** in real-world queries.

Q45. What is INNER JOIN?

Ans. INNER JOIN is used to **combine rows from two tables only when there is a match in both tables** based on a related column. If there is no matching value, those rows are not included in the result.

It works using a condition (ON clause), usually between a **primary key and a foreign key**. Only the **common data** between tables is returned.

You should use INNER JOIN when you need **only valid and matching records** from both tables. For example, fetching employees along with their department details only if the department exists.

It is useful in situations where **incomplete or unmatched data is not needed**. INNER JOIN helps in **accurate data retrieval**, ensures **data consistency**, and is widely used in real-world queries where **only related data is required**.

Q46. What is LEFT JOIN ?

Ans. LEFT JOIN (or LEFT OUTER JOIN) is used to **return all rows from the left table** and only the **matching rows from the right table**. If there is no match, the columns from the right table will be filled with **NULL values**.

It works using a condition (ON clause) to match related columns, usually **primary key to foreign key**.

You should use LEFT JOIN when you want **all records from the main (left) table**, even if related data in the second table is missing. For example, getting all employees along with their department details, including employees who are not assigned to any department.

It is very useful when you want to **identify missing relationships** or incomplete data.

LEFT JOIN is important for **complete data analysis**, helps in finding **unmatched records**, and is widely used in reporting where **no data should be missed from the main table**.

Q47. What is RIGHT JOIN ?

Ans. RIGHT JOIN (or RIGHT OUTER JOIN) is used to **return all rows from the right table** and only the **matching rows from the left table**. If there is no match, the columns from the left table will be filled with **NULL values**.

It works using a condition (ON clause) to match related columns, usually between a **primary key and a foreign key**.

You should use RIGHT JOIN when you want **all records from the right table**, even if there is no corresponding data in the left table. For example, fetching all departments along with employees, including departments that have no employees.

It is useful when the **right table is the main focus** and you don't want to miss any of its records.

RIGHT JOIN helps in **complete analysis of the right table**, identifies **missing relationships**, and ensures **no data is lost from the selected main table**.

Q48. What is FULL JOIN ?

Ans. FULL JOIN (or FULL OUTER JOIN) is used to **return all rows from both tables**, whether they match or not. If a row in one table has no match in the other, the missing side will be filled with **NULL values**.

It combines the behavior of **LEFT JOIN** and **RIGHT JOIN**, meaning it keeps all data from both tables.

You should use FULL JOIN when you need a **complete view of data from both tables**, including matched and unmatched records. It is especially useful for comparing data and finding differences between tables.

For example, you may want to see all employees and all departments, including employees without departments and departments without employees.

FULL JOIN is important for **data analysis, reconciliation, and identifying missing relationships**, as it ensures **no data is excluded from either table**.

Q49. What is CROSS JOIN ?

Ans. CROSS JOIN is used to **combine every row of one table with every row of another table**, producing a **Cartesian product**. It does not require any matching condition.

If the first table has m rows and the second has n rows, the result will have **$m \times n$ rows**.

You should use CROSS JOIN when you need **all possible combinations of two datasets**. For example, generating all combinations of products and sizes, or dates and time slots.

It is useful in scenarios like **testing, data generation, scheduling, and matrix-style reports**.

CROSS JOIN is important when you need **complete combinations of data**, but it should be used carefully because it can generate **very large result sets and impact performance**.

Q50. What is SELF JOIN?

Ans. SELF JOIN is a join where a table is **joined with itself** to compare or relate rows within the same table. It uses different aliases for the same table to treat it like two separate tables in the query.

It works with a condition (ON clause) to match rows based on a **relationship within the same table**.

You should use SELF JOIN when data in a table has a **hierarchical or recursive relationship**, such as employee-manager, parent-child, or category-subcategory.

For example, in an employees table, each employee may have a manager_id that refers to another employee in the same table.

SELF JOIN is important for handling **hierarchical data**, comparing rows within the same table, and solving problems where **relationships exist inside a single dataset**.

Q51. Difference between JOIN and UNION?

Ans. Difference between JOIN and UNION is based on **how they combine data**:

- **JOIN** → Combines data **horizontally (side by side)**
It merges columns from two or more tables based on a **related column (key)**
Used when tables have a relationship
- **UNION** → Combines data **vertically (one below another)**
It merges rows from two queries into a single result
Used when both queries have the **same number of columns and similar data types**
- **Condition requirement**
JOIN → Requires a **matching condition (ON clause)**
UNION → Does not need any condition, just combines results
- **Duplicate handling**
UNION → Removes duplicate rows by default
UNION ALL → Includes duplicates
JOIN → Does not remove duplicates automatically
- **Use case**
JOIN → When you want **related data from multiple tables**
UNION → When you want to **combine similar data from multiple queries**

Simple summary:

JOIN → combines columns

UNION → combines rows

Q52. Difference between UNION and UNION ALL?

Ans. Difference between UNION and UNION ALL is mainly about **duplicate handling and performance**:

- **UNION** → Combines results from multiple queries and **removes duplicate rows**
It returns only **unique records**
Internally, it performs an extra step to filter duplicates
- **UNION ALL** → Combines results and **keeps all rows**, including duplicates
It does not perform duplicate checking
- **Performance**
UNION → Slower because it needs to **sort and remove duplicates**
UNION ALL → Faster since it directly combines results without extra processing
- **Use case**
UNION → When you need **distinct (unique) data**
UNION ALL → When duplicates are acceptable or required and you want **better performance**
- **Requirement**
Both require same **number of columns and compatible data types**

Simple summary:

UNION → removes duplicates

UNION ALL → keeps duplicates and is faster

Q53. Real-world example of JOIN?

Ans. A real-world example of JOIN can be seen in an **e-commerce application** where data is stored in multiple related tables.

Imagine two tables:

- **customers** → stores customer details (id, name)
- **orders** → stores order details (order_id, customer_id, amount)

Here, `customer_id` in the orders table is a **foreign key** that refers to `id` in the customers table.

Now, if you want to see **which customer placed which order**, you need to combine both tables using a JOIN.

Simple example:

```
SELECT c.name, o.order_id, o.amount
FROM customers c
JOIN orders o
ON c.id = o.customer_id;
```

This query returns customer names along with their order details.

Real-world importance:

JOIN is used in almost every application like **banking, e-commerce, HR systems**, where data is split into multiple tables. It helps to **connect related data and generate meaningful information**, such as reports, dashboards, and analytics.

Q54. What is a subquery?

Ans. A subquery is a query written inside another SQL query. It is used to **perform an intermediate calculation or filtering** and then pass the result to the main query.

It is enclosed in **parentheses ()** and can be used in clauses like **SELECT, WHERE, FROM, HAVING**.

A subquery can return a **single value, multiple values, or even a table**, depending on how it is written.

It is useful when a query depends on the result of another query, such as finding data based on **dynamic conditions**.

Subqueries can be **nested (multiple levels)** and can be used with operators like **IN, EXISTS, =, >**, etc.

Subqueries help in **breaking complex problems into smaller parts** and make queries more flexible and powerful.

Q55. Types of subqueries?

Ans. Types of subqueries are classified based on **how they return data and how they are executed**:

- **Single-row subquery** → Returns **only one value (one row, one column)**
Used with operators like `=`, `>`, `<`
Example: comparing salary with average salary
- **Multiple-row subquery** → Returns **multiple values (multiple rows)**
Used with operators like `IN`, `ANY`, `ALL`
Example: finding employees in multiple departments
- **Multiple-column subquery** → Returns **more than one column**
Used when comparison is based on multiple columns
- **Correlated subquery** → Depends on the **outer query for each row**
It executes repeatedly for every row of the main query
Useful for row-by-row comparison
- **Non-correlated subquery** → Independent of the outer query
It runs **once** and its result is used by the main query

Choosing the right type of subquery is important for **correct results and performance**, especially in complex queries.

Q56. What is a correlated subquery?

Ans. A correlated subquery is a subquery that **depends on the outer query for its values**. It cannot run independently because it uses a column from the main query.

It is executed **once for each row** of the outer query, making it different from a normal (non-correlated) subquery which runs only once.

Because it runs repeatedly, it can be **slower** compared to non-correlated subqueries, especially on large datasets.

It is commonly used when you need **row-by-row comparison or filtering** based on related data.

Simple example:

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e1.department_id
);
```

This returns employees whose salary is greater than the average salary of their own department.

Correlated subquery runs **for each row of the outer query**, so it should be used carefully for performance reasons.

Q57. Difference between subquery and JOIN?

Ans. Difference between subquery and JOIN is based on **how they retrieve and combine data**:

- **Subquery** → A query inside another query used to **filter or compute values**
It returns a result that is used by the main query
Can be used in WHERE, SELECT, FROM, HAVING
- **JOIN** → Used to **combine data from multiple tables** based on a relationship
It merges columns from tables into a single result
- **Execution**
Subquery → Often executed **first (inner query)** and then outer query runs
Correlated subquery → runs multiple times (for each row)
JOIN → Executes in a **single combined operation**
- **Performance**
Subquery → Can be **slower**, especially correlated ones
JOIN → Usually **faster and more efficient** for large datasets
- **Use case**
Subquery → When you need **intermediate filtering or calculations**
JOIN → When you need **related data from multiple tables**

Simple summary:

Subquery → nested query for filtering

JOIN → combine tables for data retrieval

Q58. What is EXISTS?

Ans. The EXISTS operator is used to **check whether a subquery returns any rows**. It returns **TRUE if at least one row exists**, otherwise FALSE.

It is mainly used in the WHERE clause with a subquery to **filter data based on existence of related records**.

EXISTS does not return actual data from the subquery, it only checks **whether the result is empty or not**.

It is often used with **correlated subqueries**, where the inner query depends on the outer query.

It is usually more efficient than IN when dealing with **large datasets**, because it stops checking as soon as it finds the first match.

Simple example:

```
SELECT name  
FROM customers c  
WHERE EXISTS (  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.id  
);
```

This returns customers who have at least one order.

EXISTS is used for **existence checking**, not for comparing values, and it improves performance in many real-world scenarios.

Q59. What is NOT EXISTS?

Ans. NOT EXISTS is used to **check that a subquery returns no rows**. It returns **TRUE when the subquery is empty**, otherwise FALSE.

It is mainly used in the WHERE clause to **filter records that do not have related data** in another table.

Like EXISTS, it does not return actual data from the subquery, it only checks **whether any record exists or not**.

It is commonly used with **correlated subqueries** for row-by-row checking.

It is very useful for finding **missing or unmatched records**.

Simple example:

```
SELECT name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.id
);
```

This returns customers who have not placed any orders.

NOT EXISTS is efficient for **negative condition checking** and works well with large datasets compared to some other approaches.

Q60. What is ANY and ALL?

Ans. ANY and ALL are operators used with **subqueries** to compare a value with **multiple values returned by the subquery**.

- **ANY** → Condition is TRUE if it matches **at least one value** from the subquery
It works like OR condition
Used with operators like =, >, <, >=, <=

```
SELECT name
FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 10);
```

This returns employees whose salary is greater than **at least one employee** in department 10

- **ALL** → Condition is TRUE only if it matches **all values** from the subquery
It works like AND condition

```
SELECT name
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 10);
```

This returns employees whose salary is greater than **every employee** in department 10

Key difference:

ANY → compares with any one value

ALL → compares with all values

Important point:

ANY is less strict (one match is enough), while ALL is more strict (must satisfy all conditions).

Q61. What is a derived table?

Ans. A derived table is a **temporary result set created from a subquery** that is used in the **FROM clause** of a query. It acts like a virtual table for that query. It is not stored permanently in the database; it exists **only during query execution**.

A derived table is usually given an **alias**, because SQL requires a name to refer to it. It is useful when you need to **break complex queries into smaller parts** and then use the result as a table for further operations like JOIN, filtering, or aggregation. It can return multiple rows and columns, just like a normal table.

```
SELECT dept_id, avg_salary
FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
) AS dept_avg;
```

Here, the inner query creates a derived table called "dept_avg".

Derived tables help in **improving readability and managing complex logic**, especially in large queries.

Q62. What is CTE (Common Table Expression)?

Ans. A CTE (Common Table Expression) is a **temporary named result set** that you can define at the beginning of a query using the **WITH clause**. It works like a **temporary table** that exists only during the execution of that query.

It is similar to a derived table, but CTE is **more readable and reusable** within the same query.

You can reference a CTE multiple times, which makes complex queries **cleaner and easier to maintain**.

CTEs are often used for **breaking complex logic into steps**, performing calculations, or simplifying joins and aggregations.

They also support **recursive queries**, which are useful for hierarchical data like employee-manager relationships.

```
WITH dept_avg AS (  
    SELECT dept_id, AVG(salary) AS avg_salary  
    FROM employees  
    GROUP BY dept_id  
)  
SELECT * FROM dept_avg;
```

Here, "dept_avg" is a CTE used like a table.

CTE improves query **readability, structure, and maintainability**, especially in complex SQL queries.

Q63. Difference between CTE and subquery?

Ans. Difference between CTE and subquery is based on **structure, readability, and reuse**:

- **CTE (Common Table Expression)** → Defined using the **WITH** clause at the beginning of the query
It creates a **temporary named result set** that can be reused multiple times in the same query

- **Subquery** → Written inside another query
It is **nested** and usually used only at a specific place like WHERE, SELECT, or FROM
- **Readability**
CTE → More **readable and structured**, especially for complex logic
Subquery → Can become **hard to read** when deeply nested
- **Reusability**
CTE → Can be **referenced multiple times** in the same query
Subquery → Cannot be reused easily; must be written again
- **Execution**
CTE → Improves clarity; sometimes optimized similarly to subqueries by the database
Subquery → May be executed once or multiple times (especially correlated subqueries)
- **Use case**
CTE → Best for **complex queries, step-by-step logic, recursion**
Subquery → Best for **simple filtering or small intermediate calculations**

Simple summary:

CTE → named, reusable, readable

Subquery → nested, less reusable

Q64. What are Aggregate functions?

Ans. Aggregation is the process of **combining multiple rows of data into a single result value**. It is used when you want to **summarize data**, like totals, averages, counts, etc.

Instead of showing all individual rows, aggregation gives a **single meaningful value** based on a group of rows. It is commonly used with **GROUP BY** for grouping data.

Aggregate functions are built-in SQL functions used to perform these calculations on multiple rows. Common aggregate functions:

- **COUNT()** → Counts number of rows
- **SUM()** → Calculates total of numeric values
- **AVG()** → Calculates average value
- **MIN()** → Finds smallest value
- **MAX()** → Finds largest value

Aggregate functions **ignore NULL values** (except COUNT(*)) and always return a **single summarized result**.

Q65. What are scalar functions?

Ans. Scalar functions are functions that **operate on a single value and return a single value** as output. They do not work on multiple rows like aggregate functions.

They are used to **transform or manipulate individual data values** in a query.

Scalar functions can be applied to **each row separately**, so the result is returned for every row in the table.

Common types of scalar functions:

- **String functions** → `CONCAT()`, `UPPER()`, `LOWER()`
- **Numeric functions** → `ROUND()`, `ABS()`
- **Date functions** → `NOW()`, `CURRENT_DATE`

Scalar functions work on **single values and return single results per row**, unlike aggregate functions which work on multiple rows and return one result.

Q66. What is the window function? How does it differ from GROUP BY?

Ans. A window function is used to **perform calculations across a set of rows (called a window)** while still **returning each individual row** in the result. It does not collapse rows like aggregation.

It uses the **OVER()** clause to define the window (partition and order of data).

Common window functions include `ROW_NUMBER()`, `RANK()`, `SUM()`, `AVG()` over a **window**.

It is useful for **ranking, running totals, moving averages, and comparisons within groups**.

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS rank  
FROM employees;
```

This assigns rank to each employee based on salary.

Difference from GROUP BY:

- **GROUP BY** → Combines rows into groups and returns **one row per group**
- **Window function** → Keeps all rows and performs calculation **without reducing rows**
- **GROUP BY** → You cannot see individual row details after grouping
- **Window function** → You can see **both row data and calculated result together**

GROUP BY is used for **aggregation**, while window functions are used for **advanced analysis without losing row-level data**.

Q67. What is ROW_NUMBER()?

Ans. ROW_NUMBER() is a **window function** used to assign a **unique sequential number to each row** in a result set. The numbering starts from **1** and increases for each row. It uses the **OVER() clause** to define how rows are ordered and optionally partitioned.

The numbering is based on the **ORDER BY** inside OVER(), so the sequence depends on sorting. You can also use **PARTITION BY** to reset the numbering for each group. It is commonly used for **ranking, pagination, and removing duplicates**.

```
SELECT name, salary,  
ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM employees;
```

This assigns row numbers based on the highest salary first.

Example with partition:

```
ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC)
```

This resets numbering for each department.

ROW_NUMBER() always gives **unique numbers**, even if values are the same, unlike RANK() which can give duplicate ranks.

Q68. What is RANK()?

Ans. RANK() is a **window function** used to assign a **rank to each row** based on a specified order. It is commonly used for ranking values like salary, marks, or scores.

It uses the **OVER() clause with ORDER BY** to define how ranking is applied.

If two or more rows have the same value, they get the **same rank**, and the next rank is **skipped**.

For example, if two rows are ranked 1, the next rank will be 3 (not 2). It is useful for **leaderboards, top performers, and ranking reports**.

```
SELECT name, salary,  
RANK() OVER (ORDER BY salary DESC) AS rank  
FROM employees;
```

This assigns ranks based on salary, highest first.

RANK() allows **duplicate ranks and skips numbers**, unlike ROW_NUMBER() which always gives unique sequential numbers.

Q69. What is DENSE_RANK()?

Ans. DENSE_RANK() is a **window function** used to assign a **rank to rows based on a specified order**, similar to RANK(), but without skipping numbers.

It uses the **OVER() clause with ORDER BY** to define how ranking is applied.

If two or more rows have the same value, they get the **same rank**, but the next rank is **not skipped**.

For example, if two rows are ranked 1, the next rank will be 2 (not 3).

It is useful for **ranking data where continuous ranking is required**, like leaderboards or scoring systems.

```
SELECT name, salary,  
DENSE_RANK() OVER (ORDER BY salary DESC) AS rank  
FROM employees;
```

This assigns ranks based on salary without gaps.

DENSE_RANK() gives **same rank for duplicates but maintains continuous sequence**, unlike RANK() which skips numbers.

Q70. What is PARTITION BY?

Ans. PARTITION BY is used in **window functions** to **divide the result set into smaller groups (partitions)** and then apply the function **within each group separately**.

It works like GROUP BY in terms of grouping, but it **does not reduce rows**. Each row is still returned, but calculations are done within its partition.

It is used inside the **OVER() clause** along with functions like ROW_NUMBER(), RANK(), SUM(), AVG(), etc.

When you use PARTITION BY, the calculation **restarts for each group**.

```
SELECT name, department_id, salary,  
ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS row_num  
FROM employees;
```

Here, row numbers restart for each department.

Difference from GROUP BY:

PARTITION BY → keeps all rows and applies calculation per group

GROUP BY → combines rows and returns one row per group

PARTITION BY is useful for **group-based analysis while still keeping row-level data**.

Q71. What is LEAD() and LAG()?

Ans. LEAD() and LAG() are **window functions** used to access values from **other rows in the same result set** without using a JOIN.

- **LAG()** → Gets the value from a **previous row** (above the current row)
- **LEAD()** → Gets the value from a **next row** (below the current row)

They are used with the **OVER() clause**, usually with ORDER BY to define the row sequence.

You can also specify how many rows forward or backward to move, and a default value if no row exists.

They are very useful for **comparing rows, finding differences, trends, and time-based analysis**.

```
SELECT name, salary,  
LAG(salary) OVER (ORDER BY salary) AS prev_salary,  
LEAD(salary) OVER (ORDER BY salary) AS next_salary  
FROM employees;
```

This shows previous and next salary for each employee.

LEAD() looks **forward**, LAG() looks **backward**, and both help in **row-by-row comparison without self joins**.

Q72. What is NTILE()?

Ans. NTILE() is a **window function** used to **divide rows into a specified number of equal groups (buckets)** and assign a group number to each row.

It is mainly used for **data distribution and ranking into categories**, like quartiles (4 groups), deciles (10 groups), etc.

It works with the **OVER() clause and ORDER BY**, which decides how rows are sorted before dividing into groups.

The groups are numbered starting from **1 up to N** (the number you specify).

If rows cannot be divided equally, some groups may have **one extra row**.

```
SELECT name, salary,  
NTILE(4) OVER (ORDER BY salary DESC) AS quartile  
FROM employees;
```

This divides employees into 4 groups based on salary.

NTILE() is useful for **ranking data into segments** like top 25%, middle 50%, etc., and is commonly used in **analytics and reporting**.

Q73. Real-world use of window functions?

Ans. Window functions are widely used in real-world applications for **analysis, reporting, and decision-making**, especially when you need calculations without losing row-level data.

- **Ranking and leaderboards** → Used in apps like gaming or sales dashboards to rank users or employees
Example: top 10 salespersons using `RANK()` or `DENSE_RANK()`
- **Pagination** → Used in websites to display data page by page
Example: using `ROW_NUMBER()` to fetch rows 1–10, 11–20, etc.
- **Running totals (cumulative sum)** → Used in finance or banking to track balance over time
Example: calculating daily account balance using `SUM() OVER()`
- **Comparing previous and next values** → Used in stock market or analytics
Example: using `LAG()` to compare today's price with yesterday's price
- **Grouping and segmentation** → Used in marketing to divide customers into categories
Example: using `NTILE()` to create top, middle, and low-value customer groups
- **Finding duplicates or latest records** → Used in data cleaning
Example: using `ROW_NUMBER()` with partition to identify duplicate entries

Window functions are powerful because they allow **advanced analysis (ranking, trends, comparisons)** while still keeping **all original rows visible**, which is not possible with GROUP BY alone.

Q74. Types of normal forms?

Ans. Types of normal forms are rules used in **database normalization** to organize data and reduce redundancy. The most commonly used normal forms are:

- **First Normal Form (1NF)** → Ensures that all columns have **atomic (single) values**
No repeating groups or multiple values in one column
Each record should be unique

- **Second Normal Form (2NF)** → Must be in 1NF and removes **partial dependency**
All non-key columns must depend on the **entire primary key** (important for composite keys)
- **Third Normal Form (3NF)** → Must be in 2NF and removes **transitive dependency**
Non-key columns should depend **only on the primary key**, not on other non-key columns
- **Boyce-Codd Normal Form (BCNF)** → Advanced version of 3NF
Ensures that **every determinant is a candidate key**
Handles some edge cases not covered by 3NF
- **Fourth Normal Form (4NF)** → Removes **multi-valued dependencies**
Ensures independent multi-valued facts are stored in separate tables
- **Fifth Normal Form (5NF)** → Removes **join dependency**
Ensures data cannot be further decomposed without losing information

Important point:

In most real-world applications, normalization is usually applied up to **3NF or BCNF**, as higher forms are rarely needed.

Q75. What are relationships in DB?

Ans. Relationships in a database define **how tables are connected to each other** based on common columns. They help organize data across multiple tables and allow you to **retrieve related information easily**.

Relationships are usually created using **primary key and foreign key** constraints.

Main types of relationships:

- **One-to-One (1:1)** → One record in Table A is related to **only one record** in Table B
Example: one person → one passport
- **One-to-Many (1:N)** → One record in Table A is related to **multiple records** in Table B
Example: one department → many employees
- **Many-to-Many (M:N)** → Multiple records in Table A are related to **multiple records** in Table B

Implemented using a **junction (bridge) table**

Example: students and courses

These relationships help in **data normalization, avoiding redundancy, and maintaining data integrity.**

Important point:

Relationships are the core of relational databases, allowing data to be **connected, structured, and efficiently queried** using JOINS.

Q76. Types of relationships (1-1, 1-M, M-M)?

Ans. Types of relationships in a database define **how records in one table are connected to records in another table.** The main types are:

- **One-to-One (1:1)** → One record in Table A is related to **only one record** in Table B, and vice versa
Example: one person has one passport
Used when data is split for **security or organization purposes**
- **One-to-Many (1:M)** → One record in Table A is related to **multiple records** in Table B
Example: one department has many employees
This is the **most common relationship** in databases
- **Many-to-Many (M:M)** → Multiple records in Table A are related to **multiple records** in Table B
Example: students can enroll in many courses, and courses can have many students
This is implemented using a **junction (bridge) table** that contains foreign keys from both tables

Important point:

These relationships help in **structuring data efficiently, reducing redundancy, and enabling powerful queries** using JOINS.

Q77. What is an ER diagram?

Ans. An ER diagram (Entity-Relationship diagram) is a **visual representation of a database structure**. It shows how different **entities (tables)** are related to each other.

It is mainly used during the **database design phase** to plan and organize data clearly before creating actual tables.

Key components:

- **Entity** → Represents a table (example: Employee, Student)
- **Attribute** → Represents columns of a table (example: name, id, salary)
- **Relationship** → Shows how entities are connected (1:1, 1:M, M:M)

Symbols used:

- Rectangle → Entity
- Oval → Attribute
- Diamond → Relationship

Example:

Student — enrolls — Course

This shows students and courses are related.

ER diagrams help in **understanding database structure, reducing errors, improving design, and communicating ideas clearly** among developers.

ER diagram is a **blueprint of the database**, used before implementation to ensure proper structure and relationships.

Q78. What is schema vs database?

Ans. Difference between schema and database is based on **structure vs container**:

- **Database** → A database is a **collection of related data** stored in an organized way. It includes tables, views, indexes, procedures, etc. It acts as a **container that holds everything**

- **Schema** → A schema is the **logical structure or blueprint** of the database
It defines how data is organized, including **tables, columns, data types, and relationships**
- **Relationship between them**
A database can have **multiple schemas**
Each schema organizes objects inside the database
- **Purpose**
Database → stores actual data
Schema → defines **how that data is structured**
- **Example for clarity**
Database = complete library
Schema = sections inside the library (science, history, etc.)

Database is the **physical container**, while schema is the **design/structure inside it**.

Q79. What is data integrity?

Ans. Data integrity means **ensuring that data in a database remains accurate, consistent, and reliable** throughout its lifecycle. It focuses on maintaining the **correctness and trustworthiness of data** during operations like insert, update, and delete.

- **Accuracy** → Data should be correct and valid
- **Consistency** → Data should remain uniform across all tables and transactions
- **Reliability** → Data should not get corrupted over time

Types of data integrity:

- **Entity Integrity** → Ensures each row is uniquely identified using a **primary key** (no duplicate or NULL primary keys)
- **Referential Integrity** → Ensures relationships between tables are valid using **foreign keys** (no orphan records)
- **Domain Integrity** → Ensures data follows rules like **data type, range, and format** using constraints (NOT NULL, CHECK, DEFAULT)
- **User-defined Integrity** → Custom business rules applied based on application requirements

Data integrity is critical because it ensures **correct query results, prevents invalid or duplicate data, and maintains overall database quality.**

Q80. What is a transaction?

Ans. A transaction is a **logical unit of work in a database** that consists of one or more SQL operations executed as a **single unit**. The key idea is: either **all operations succeed or none are applied**.

A transaction ensures that the database moves from one **consistent state to another** without leaving partial updates.

Core properties (ACID properties):

- **Atomicity** → All steps complete or the entire transaction is rolled back
- **Consistency** → Data remains valid before and after the transaction
- **Isolation** → Concurrent transactions do not interfere with each other
- **Durability** → Once committed, changes are permanently saved

Common transaction control commands:

- **BEGIN / START TRANSACTION** → Starts a transaction
- **COMMIT** → Saves all changes permanently
- **ROLLBACK** → Undoes all changes in case of error
- **SAVEPOINT** → Creates a checkpoint within a transaction

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;
```

This transfers money from one account to another as a single unit.

Transactions are essential for **data integrity and reliability**, especially in real-world systems like banking, where partial updates can cause serious issues.

Q81. What are ACID properties?

Ans. ACID properties are a set of rules that ensure **reliable and safe execution of database transactions**. They guarantee that a transaction behaves correctly even in case of failures or concurrent access.

- **Atomicity** → A transaction is treated as a **single unit**
Either all operations are completed or none are applied
If any step fails, the entire transaction is rolled back
- **Consistency** → Ensures the database moves from one **valid state to another valid state**
All rules, constraints, and relationships must be satisfied after the transaction
- **Isolation** → Multiple transactions running at the same time **do not affect each other**
Intermediate changes are not visible to other transactions
- **Durability** → Once a transaction is committed, the changes are **permanently stored**
Data will not be lost even if there is a system failure

These properties together form the ACID properties model used in DBMS.

ACID ensures **data integrity, consistency, and fault tolerance**, which is critical in real-world systems like banking and financial applications.

Q82. What is COMMIT?

Ans. COMMIT is a transaction control command used to **permanently save all changes** made during the current transaction to the database.

Once COMMIT is executed, the changes become **final and visible to other users**, and they **cannot be undone using ROLLBACK**.

It marks the successful completion of a transaction and ensures **durability** (from the ACID properties).

Used after successful execution of all operations in a transaction. Makes all INSERT, UPDATE, DELETE changes permanent. Releases locks held by the transaction

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
COMMIT;
```

Here, the balance update is permanently saved after COMMIT.

COMMIT confirms that **all changes are correct and should be stored permanently** in the database.

Q83. What is ROLLBACK?

Ans. ROLLBACK is a transaction control command used to **undo all changes made in the current transaction** and restore the database to its **previous consistent state**.

It is typically used when an error occurs or when you decide not to proceed with the transaction. All uncommitted changes are **discarded completely**.

- Reverts INSERT, UPDATE, DELETE operations done after the transaction started
- Works only for changes that are **not yet committed**
- Helps maintain **data consistency and integrity**

It supports **partial rollback** using SAVEPOINT, where you can roll back to a specific checkpoint instead of the beginning.

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
ROLLBACK;
```

Here, the balance update is undone, and no change is saved.

ROLLBACK ensures **Atomicity**, meaning either all operations succeed or none are applied.

ROLLBACK is used as a **safety mechanism** to prevent incorrect or partial data from being saved.

Q84. What is SAVEPOINT?

Ans. SAVEPOINT is a transaction control feature used to **create a checkpoint inside a transaction**. It allows you to **roll back only part of a transaction** instead of undoing everything.

When you set a SAVEPOINT, the database remembers that point. Later, you can use ROLLBACK TO SAVEPOINT to **undo changes made after that point**, while keeping earlier changes intact.

- Used within a transaction (after BEGIN)
- Helps in handling **complex transactions with multiple steps**
- Allows **partial rollback instead of full rollback**

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
SAVEPOINT sp1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
ROLLBACK TO sp1;  
COMMIT;
```

Here, the second update is undone, but the first update remains.

SAVEPOINT helps maintain **Atomicity** by giving controlled rollback within a transaction.

SAVEPOINT is useful when you want **flexibility in error handling without losing all progress in a transaction**.

Q85. What is indexing?

Ans. Indexing is the technique of creating a **data structure on one or more columns** to **speed up data retrieval** from a table. Instead of scanning the entire table, the database uses the index to **quickly locate the required rows**.

It works similarly to an index in a book, where you can jump directly to the needed page instead of reading everything.

Improves performance of queries, especially with **WHERE, JOIN, ORDER BY, GROUP BY**. Stores **sorted references (pointers)** to actual data rows. Can be created on **single or multiple columns**

Types include clustered and non-clustered indexes, depending on how data is stored and accessed.

However, indexing has trade-offs:

- **Faster reads (SELECT)**
- **Slower writes (INSERT, UPDATE, DELETE)** because indexes also need to be updated

Simple example:

```
CREATE INDEX idx_name ON employees(name);
```

Indexing is crucial for **performance optimization**, but it should be used carefully to balance **speed and storage/maintenance cost**.

Q86. What types of SQL operators do you know?

Ans. SQL operators are symbols or keywords used to **perform operations on data and define conditions** in queries. They are broadly classified as follows:

- **Arithmetic Operators** → Perform mathematical calculations
+ (addition), - (subtraction), * (multiplication), / (division), % (modulus)
- **Comparison (Relational) Operators** → Compare values and return TRUE/FALSE
= (equal), != or <> (not equal), > (greater than), < (less than), >=, <=
- **Logical Operators** → Combine multiple conditions
AND, OR, NOT
- **Special Operators** → Used for specific filtering conditions
IN, BETWEEN, LIKE, IS NULL, EXISTS, ANY, ALL
- **Set Operators** → Combine results of multiple queries
UNION, UNION ALL, INTERSECT, EXCEPT
- **Assignment Operator** → Assigns values (used in some SQL dialects)
= (for setting values in UPDATE statements)
- **Bitwise Operators** (supported in some databases) → Work on binary values
&, |, ^

Operators are essential for **filtering, comparing, combining, and manipulating data**, and they form the core logic of SQL queries.

Q87. What is a clause?

Ans. A clause in SQL is a **part of a query that performs a specific function**. It is a building block of a SQL statement, and multiple clauses are combined to form a complete query. Each clause controls **how data is selected, filtered, grouped, or sorted**.

Common clauses:

- **SELECT** → Specifies which columns to retrieve
- **FROM** → Specifies the table(s) to read data from
- **WHERE** → Filters rows based on conditions
- **GROUP BY** → Groups rows for aggregation
- **HAVING** → Filters grouped data (after aggregation)
- **ORDER BY** → Sorts the result set
- **LIMIT / OFFSET** → Restricts number of rows returned

Clauses are written in a specific logical order to form a valid query.

A clause defines **what operation should be performed in a query**, and combining clauses correctly is essential for writing efficient SQL queries.

Q88. What is VIEW ?

Ans. A VIEW is a **virtual table defined by a stored query**. It does not store data physically (in most cases); instead, it **displays data dynamically** from one or more underlying tables whenever you query it.

You create a view by saving a SELECT statement, and then you can query the view **like a regular table**.

- Used to **simplify complex queries** by encapsulating joins, filters, and calculations
- Helps with **security** by exposing only selected columns/rows to users
- Always shows **up-to-date data** from base tables (unless it's a materialized view)
- Can be **simple (single table)** or **complex (multiple tables with joins/aggregations)**

A view is mainly a **logical abstraction layer** that improves readability, reuse, and access control without duplicating data.

Q89. What is Stored Procedure ?

Ans. A stored procedure is a **precompiled set of SQL statements stored in the database** that can be executed as a single unit. It is used to **encapsulate business logic** inside the database.

Instead of writing the same SQL repeatedly, you define it once as a procedure and call it whenever needed.

- Improves **performance** because it is precompiled and optimized
- Enhances **reusability** by avoiding repeated code
- Provides **security** by restricting direct table access
- Supports **input and output parameters**
- Can include control logic like IF conditions, loops, and error handling

```
CREATE PROCEDURE GetEmployees()  
BEGIN  
    SELECT * FROM employees;  
END;
```

Calling the procedure:

```
CALL GetEmployees();
```

Stored procedures help in **modularizing database logic**, improving performance, and maintaining cleaner application code.

Q90. How to delete a table from a database?

Ans. To delete a table from a database, you use the **DROP TABLE** command. This removes the table **completely**, including its structure and all data.

- It is a **DDL operation** (Data Definition Language)
- The action is **permanent** and cannot be rolled back in most systems
- All indexes, constraints, and data in that table are also removed

Basic syntax:

```
DROP TABLE table_name;
```

- Make sure the table is not required before dropping it
- If other tables depend on it (via foreign keys), the operation may fail or require **CASCADE** (depends on DBMS)
- If you only want to remove data but keep the table structure, use **DELETE** or **TRUNCATE** instead

Simple summary:

DROP TABLE → removes **table + data permanently**

Q91. What SQL scalar functions do you know?

Ans. Scalar functions operate on a single input value and return a single output value per row. They're commonly used for **data transformation, formatting, and calculations** within queries.

String functions (text processing)

- **UPPER()**, **LOWER()** → change case
- **LENGTH()** / **LEN()** → string length (dialect-specific)
- **SUBSTRING()** / **SUBSTR()** → extract part of a string
- **CONCAT()** → join strings
- **TRIM()**, **LTRIM()**, **RTRIM()** → remove spaces
- **REPLACE()** → replace part of a string

Numeric functions (math operations)

- **ROUND()**, **CEIL()** / **CEILING()**, **FLOOR()** → control numeric precision
- **ABS()** → absolute value
- **POWER()** → exponentiation
- **MOD()** → remainder

Date and time functions

- **CURRENT_DATE**, **CURRENT_TIMESTAMP** → current date/time
- **NOW()** (MySQL/PostgreSQL) → current timestamp
- **DATEADD()**, **DATEDIFF()** (SQL Server) → add/compare dates
- **EXTRACT()** → get parts like year, month, day

Conversion functions (type casting)

- `CAST()` → convert one data type to another
- `CONVERT()` → alternative casting (SQL Server/MySQL)

Conditional functions

- `COALESCE()` → returns first non-NULL value
- `NULLIF()` → returns NULL if two values are equal
- `CASE` → conditional logic inside queries

Scalar functions are applied **row by row**, and excessive use (especially in WHERE clauses) can impact performance by **preventing index usage**.

Q92. What are SQL set operators?

Ans. SQL set operators are used to **combine the results of two or more SELECT queries** into a single result set. The queries involved must return the **same number of columns with compatible data types**.

Common SQL set operators:

- **UNION** → Combines results and **removes duplicate rows**
Ensures only unique records are returned
- **UNION ALL** → Combines results and **keeps all rows**, including duplicates
Faster than UNION because it skips duplicate checking
- **INTERSECT** → Returns only the **common rows present in both queries**
- **EXCEPT** (or MINUS in some databases) → Returns rows from the first query that are **not present in the second query**

Important points:

- Column order and data types must match across queries
- ORDER BY is applied only to the **final combined result**
- Use UNION ALL when duplicates are acceptable for **better performance**

Set operators combine **multiple query results into one** based on specific rules.

Q93. What is the difference between the DELETE and TRUNCATE statements?

Ans. Difference between DELETE and TRUNCATE is based on how they remove data, performance, and transaction behavior:

- **DELETE** → Removes **specific rows or all rows** from a table
You can use a **WHERE clause** to filter which rows to delete
It is a **DML (Data Manipulation Language)** operation
- **TRUNCATE** → Removes **all rows from a table** at once
You **cannot use WHERE**; it always deletes the entire table data
It is a **DDL (Data Definition Language)** operation
- **Performance**
DELETE → Slower because it deletes rows **one by one**
TRUNCATE → Faster because it **deallocates data pages**
- **Rollback**
DELETE → Can be **rolled back** before COMMIT
TRUNCATE → Usually **cannot be rolled back** (depends on DBMS)
- **Logging**
DELETE → Fully logged (each row operation is recorded)
TRUNCATE → Minimally logged
- **Auto-increment / Identity**
DELETE → Does not reset identity values
TRUNCATE → Resets identity/auto-increment counter
- **Triggers**
DELETE → Fires DELETE triggers
TRUNCATE → Does not fire triggers

Use DELETE when you need **controlled row removal**, and TRUNCATE when you need **fast clearing of entire table data**.

Q94. What is the difference between the DROP and TRUNCATE statements?

Ans. The difference between DROP and TRUNCATE is about **what gets removed** and **how permanent the action is**:

- **TRUNCATE** → Removes **all rows (data only)** from a table
The **table structure remains intact** (columns, indexes, constraints stay)
It is fast because it **deallocates data pages**

Resets **auto-increment/identity values**

Cannot use WHERE; always clears the whole table

- **DROP** → Removes the **entire table itself**
Deletes **structure + data + indexes + constraints**
After DROP, the table **no longer exists** in the database
- **Type of command**
TRUNCATE → DDL operation (though it affects data)
DROP → DDL operation
- **Rollback**
TRUNCATE → Usually not reversible (DBMS-dependent)
DROP → Not reversible; table must be recreated

```
TRUNCATE TABLE employees; -- clears all data, keeps table
```

```
DROP TABLE employees; -- deletes table completely
```

TRUNCATE = **clear data, keep table**

DROP = **delete table completely**

Q95. What is the ALTER command?

Ans. The **ALTER** command is a Data Definition Language (DDL) statement used to **modify the structure of an existing database object**, most commonly a table. It lets you change how a table is defined **without deleting and recreating it**.

What you can do with ALTER:

- **Add a new column**

```
ALTER TABLE employees
```

```
ADD age INT;
```

- **Modify an existing column (type/size/constraints)**
(Syntax varies by DBMS)

```
ALTER TABLE employees
```

```
MODIFY salary DECIMAL(10,2);
```

- **Rename a column**

```
ALTER TABLE employees  
RENAME COLUMN name TO full_name;
```

- **Drop (remove) a column**

```
ALTER TABLE employees  
DROP COLUMN age;
```

- **Add constraints**

```
ALTER TABLE employees  
ADD CONSTRAINT pk_emp PRIMARY KEY (id);
```

- **Drop constraints**

```
ALTER TABLE employees  
DROP CONSTRAINT pk_emp;
```

- **Rename a table**

```
ALTER TABLE employees  
RENAME TO staff;
```

Key considerations:

- It changes **schema (structure)**, not just data
- May require **locks**, impacting concurrent access
- Some operations can be **costly** on large tables (e.g., type changes)
- Syntax differs slightly across databases (MySQL, PostgreSQL, SQL Server)

ALTER is used to **evolve your schema safely**—adding, changing, or removing columns and constraints—without recreating the table.

Q96. What is the execution order of SQL commands?

Ans. The execution order of SQL commands (more precisely, **logical query processing order**) defines **how the database actually evaluates a SELECT query**, which is different from the order you write it.

Logical execution order:

1. **FROM** → Identify source tables and perform joins
 2. **ON** → Apply join conditions
 3. **JOIN** → Combine rows from tables
 4. **WHERE** → Filter rows (before grouping)
 5. **GROUP BY** → Group rows for aggregation
 6. **HAVING** → Filter grouped data
 7. **SELECT** → Choose columns and apply expressions/aliases
 8. **DISTINCT** → Remove duplicate rows
 9. **ORDER BY** → Sort the final result
 10. **LIMIT / OFFSET** → Restrict number of rows returned
- Even though **SELECT** appears first, it is processed **after WHERE, GROUP BY, HAVING**
 - Aliases defined in **SELECT** are **not available in WHERE** (because **SELECT** runs later)
 - Understanding this order helps avoid logical errors and improves query writing

SQL is written **top-down**, but executed in a **different logical order**, which is crucial for correct results and optimization.

Q97. What is the difference between SQL and SQLite?

Ans. The difference between SQL and SQLite is about **language vs database system**:

- **SQL** (Structured Query Language) is a **standard language** used to **query, insert, update, and manage data** in relational databases.
It is not a database itself; it is used by many systems like MySQL, PostgreSQL, and Oracle Database.

- **SQLite** is a **lightweight relational database management system (RDBMS)** that **uses SQL** to interact with data. It is a **software library**, not a server-based system.

Key differences:

- **Nature**
SQL → Query language
SQLite → Database engine (software)
- **Setup**
SQL → No installation by itself (it's just a language)
SQLite → No server needed; **embedded database** inside applications
- **Architecture**
SQL → Used in client-server databases
SQLite → **File-based**, stores data in a single **.db** file
- **Use cases**
SQL → Used across all relational databases
SQLite → Best for **small apps, mobile apps, local storage, testing**
- **Scalability**
SQL → Used in large, scalable systems
SQLite → Limited scalability, not ideal for high-concurrency systems

SQL is the **language**, while SQLite is a **database system** that uses that language.

Q98. What is a Datawarehouse?

Ans. A data warehouse is a **centralized repository designed to store and analyze large volumes of historical data** from multiple sources. It is optimized for **reporting, analytics, and decision-making**, not for day-to-day transaction processing.

Core characteristics:

- **Subject-oriented** → Organized around business domains (sales, finance, customers)
- **Integrated** → Combines data from different systems into a consistent format
- **Time-variant** → Stores historical snapshots for trend analysis
- **Non-volatile** → Data is mostly **read-only** (loaded, not frequently updated/deleted)

Typical flow:

- Data is extracted from source systems, transformed, and loaded via **ETL**
- Stored in a structured schema (often **star/snowflake**)
- Queried using SQL for dashboards, BI, and analytics

Components:

- **Staging area** (raw ingest)
- **Warehouse storage** (fact & dimension tables)
- **Data marts** (department-specific subsets)
- **Metadata & governance** layers

Example platforms:

- Amazon Redshift
- Google BigQuery
- Snowflake

How it differs from an OLTP database:

- OLTP → optimized for **fast inserts/updates** (transactions)
- Data warehouse → optimized for **complex queries and aggregations** (analytics)

A data warehouse enables **fast, reliable analytics over integrated historical data**, supporting business intelligence, reporting, and strategic decisions.

Q99. What is the difference between cross join and natural join?

Ans. The difference between CROSS JOIN and NATURAL JOIN is mainly about **how tables are combined and whether a condition is used**:

- **CROSS JOIN** → Produces a **Cartesian product**
Every row from the first table is combined with **every row from the second table**. No condition is required
Result size = rows in table A × rows in table B

Example:

```
SELECT *  
FROM A  
CROSS JOIN B;
```

- **NATURAL JOIN** → Automatically joins tables based on **columns with the same name and compatible data types**
No need to explicitly write a join condition
It behaves like an INNER JOIN using those common columns

Example:

```
SELECT *  
FROM A  
NATURAL JOIN B;
```

Key differences:

- **Condition**
CROSS JOIN → No condition
NATURAL JOIN → Implicit condition based on common columns
- **Result size**
CROSS JOIN → Very large (all combinations)
NATURAL JOIN → Only matching rows
- **Usage**
CROSS JOIN → Used when you need **all combinations**
NATURAL JOIN → Used when tables share **common column names**

CROSS JOIN can generate **huge datasets**, while NATURAL JOIN depends on **column name matching**, which can sometimes lead to **unexpected results** if not used carefully.

Q100. What do you mean by “Trigger” in SQL?

Ans. A trigger is a special type of database object that automatically executes (fires) in response to specific events on a table or view. You don't call it manually; the database invokes it when the defined event occurs.

Typical events:

- **INSERT** → after/before a new row is added
- **UPDATE** → after/before a row is modified
- **DELETE** → after/before a row is removed

Key characteristics:

- Can be **BEFORE** or **AFTER** the event (timing)
- Can be **row-level** (fires for each row) or **statement-level** (fires once per statement)
- Runs inside the same transaction as the triggering statement

Common uses:

- **Auditing/logging** changes (who changed what, when)
- **Enforcing business rules** that are hard to express with constraints
- **Maintaining derived data** (e.g., updating summary tables)
- **Data validation** and automatic corrections

```
CREATE TRIGGER trg_after_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log(emp_id, action)
    VALUES (NEW.id, 'INSERT');
END;
```

Here, whenever a row is inserted into **employees**, a record is automatically written to **audit_log**.

Important points:

- Triggers can **impact performance** if they do heavy work
- They can make behavior **less visible** (logic runs implicitly), so document them well

- Prefer constraints and application logic when possible; use triggers for **cross-cutting concerns** like auditing

In short: a trigger is **event-driven logic inside the database** that enforces rules or performs side effects automatically.

Q101. What is the default ordering of data using the ORDER BY clause? How could it be changed?

Ans. By default, when you use the ORDER BY clause in SQL, the data is sorted in **ascending order**.

- **Ascending (ASC)** means:
 - Numbers → smallest to largest
 - Strings → A to Z
 - Dates → oldest to newest

Even if you don't specify it, SQL assumes:

ORDER BY column_name ASC;

How to change the order:

You can explicitly specify the sorting direction using:

- **ASC** → Ascending (default)
- **DESC** → Descending (reverse order)

```
-- Default (ascending)
SELECT * FROM employees
ORDER BY salary;

-- Explicit ascending
SELECT * FROM employees
ORDER BY salary ASC;

-- Descending
SELECT * FROM employees
ORDER BY salary DESC;
```

You can also sort by multiple columns:

```
SELECT * FROM employees
ORDER BY department ASC, salary DESC;
```

Here:

- First sorts by department (A–Z)
- Then within each department, sorts salary from high to low

ORDER BY sorts data **ascending by default**, and you can change it using **ASC** or **DESC** based on your requirement.

Q102. What is SQL Injection?

Ans. SQL injection is a **security vulnerability** where an attacker inserts malicious SQL code into an application's input fields to **manipulate database queries**. It exploits cases where user input is **directly concatenated into SQL statements** without proper validation or parameterization.

Common impacts:

- Unauthorized **data access** (read sensitive records)
- **Authentication bypass**
- **Data modification/deletion** (INSERT/UPDATE/DELETE)
- In some cases, **database takeover** or remote command execution

SQL injection is fundamentally about **mixing code and untrusted data**. Keep them separate using **parameterized queries**, and you eliminate the primary attack vector.

Q103. What is OLTP?

Ans. OLTP (Online Transaction Processing) refers to systems designed to **manage and process large numbers of short, real-time transactions**. These systems focus on **fast inserts, updates, and deletes** while maintaining data accuracy.

Typical examples include **banking systems, e-commerce order processing, ATM transactions, and reservation systems**, where many users interact with the database simultaneously.

Key characteristics:

- **High transaction volume** → Handles thousands of small operations per second
- **Real-time processing** → Immediate response to user actions
- **Data consistency** → Strong reliance on ACID properties
- **Normalized data** → Data is structured to reduce redundancy
- **Short queries** → Simple queries affecting a small number of rows
-

OLTP systems are optimized for **speed, concurrency, and reliability of transactions**, not for heavy data analysis.

Q104. What is OLAP?

Ans. OLAP (Online Analytical Processing) refers to systems designed for **analyzing large volumes of data** to support **reporting, business intelligence, and decision-making**.

Unlike transactional systems, OLAP focuses on **complex queries, aggregations, and trends over time**, often using historical data from a **data warehouse**.

Key characteristics:

- **Read-intensive workloads** → Mostly SELECT queries
- **Complex queries** → Involve aggregations like SUM, AVG, COUNT
- **Historical data analysis** → Works on large, time-based datasets
- **Denormalized structures** → Often uses star or snowflake schemas for faster querying

- **Low update frequency** → Data is periodically loaded, not frequently modified

Typical operations in OLAP:

- **Roll-up** → Aggregate data (e.g., daily → monthly sales)
- **Drill-down** → View detailed data (e.g., yearly → daily)
- **Slice and dice** → Filter and view data from different perspectives

Difference from OLTP:

- OLAP → Used for **analysis and reporting**
- OLTP → Used for **day-to-day transactions**

OLAP systems are optimized for **fast analytical queries on large datasets**, enabling better insights and business decisions.

Q105. What is a pivot table in SQL?

Ans. A pivot table in SQL is a way to **transform row data into columns**, making it easier to **summarize and analyze data** in a cross-tabular format. It's commonly used for **reporting**, similar to pivot tables in spreadsheet tools.

In SQL, this transformation is often done using a **PIVOT operation** (available in some databases like SQL Server) or by using **conditional aggregation** in others.

Concept:

You take distinct values from one column and turn them into **separate columns**, applying an aggregate function (like SUM, COUNT) on another column.

Key points:

- Converts **rows** → **columns**
- Uses **aggregate functions** like SUM, COUNT
- Useful for **reports and dashboards**

A pivot table helps in **summarizing data in a more readable, column-based format**, especially for comparisons across categories.

Q106. What is COALESCE()?

Ans. COALESCE() is a SQL function that returns the **first non-NULL value** from a list of expressions. It's primarily used to **handle missing (NULL) data** in queries.

Syntax:

```
COALESCE(expr1, expr2, expr3, ..., exprN)
```

- Evaluates arguments **left to right**
- Returns the **first value that is not NULL**
- If all are NULL, the result is **NULL**

Why use COALESCE():

- Clean handling of NULLs without verbose **CASE** expressions
- Portable across databases (standard SQL)
- Helps produce **complete, user-friendly outputs**

COALESCE() doesn't change stored data—it only **controls how NULLs are handled in query results**.

Q107. How do you handle NULLs in aggregations?

Ans. Handling **NULL** values in SQL aggregations is important because **most aggregate functions ignore NULLs by default**. If you don't account for that behavior, results can be misleading.

Default behavior:

- **SUM()**, **AVG()**, **MIN()**, **MAX()** → **ignore NULLs**
- **COUNT(column)** → counts only **non-NULL values**
- **COUNT(*)** → counts **all rows**, including those with NULLs

Example:

```
SELECT COUNT(salary), COUNT(*) FROM employees;
```

- **COUNT(salary)** → counts only rows where salary is not NULL
- **COUNT(*)** → counts all rows

Techniques to handle NULLs explicitly:

1. Replace NULLs using COALESCE()

```
SELECT SUM(COALESCE(salary, 0)) FROM employees;
```

- Treats NULL as 0 during aggregation

2. Use IFNULL() (SQLite/MySQL specific)

```
SELECT AVG(IFNULL(salary, 0)) FROM employees;
```

3. Control NULL behavior with CASE

```
SELECT SUM(CASE WHEN salary IS NULL THEN 0 ELSE salary END)  
FROM employees;
```

4. Filter NULLs explicitly

```
SELECT AVG(salary)  
FROM employees  
WHERE salary IS NOT NULL;
```

5. Count NULLs separately (if needed)

```
SELECT COUNT(*) - COUNT(salary) AS null_count  
FROM employees;
```

Important considerations:

- Replacing NULL with 0 changes business meaning (0 ≠ missing value)
- For **AVG()**, replacing NULLs can **distort averages**
- Sometimes it's better to **exclude NULLs instead of replacing them**

By default, aggregations **ignore NULLs**, so you must decide whether to **exclude them or substitute them** based on the business logic.

Q108. What do you mean by buffer pool and mention its benefits?

Ans. A buffer pool is a region of main memory (RAM) used by a database engine to cache data pages that are read from disk. Instead of hitting the disk for every query, the DBMS serves data from this in-memory cache whenever possible.

How it works (at a high level):

- Data on disk is stored in fixed-size **pages** (e.g., 4KB/8KB).
- When a query needs a page, the engine loads it into the buffer pool.
- Subsequent requests for the same page are served from memory (**cache hit**).
- If memory is full, a replacement policy (e.g., LRU-like) evicts less-used pages.
- Modified pages (“dirty pages”) are **written back to disk later** (checkpoint/flush).

Benefits:

- **Much faster reads**
Memory access is orders of magnitude faster than disk I/O, so cache hits drastically reduce latency.
- **Reduced disk I/O**
Frequently accessed data stays in RAM, lowering the number of physical reads/writes.
- **Better throughput and scalability**
More queries can be served concurrently because the system isn’t bottlenecked by disk.
- **Write optimization**
Changes can be **batched** and flushed efficiently (write-behind), reducing random writes.
- **Improved CPU efficiency**
Less waiting on I/O means CPUs spend more time doing useful work.
- **Support for concurrency control**
The buffer pool often integrates with **latching/locking** on pages, aiding safe concurrent access.

Sizing and tuning the buffer pool (e.g., giving it a large portion of available RAM in systems like MySQL InnoDB or PostgreSQL shared buffers) has a **major impact on query performance**, often more than many query-level optimizations.

Q109. What is Database Cardinality?

Ans. Database cardinality refers to the **number of unique values** in a column (attribute) of a table. It's a measure of how **distinct** the data is.

- **High cardinality** → Many unique values
Example: `email`, `employee_id` (almost every row is different)
- **Low cardinality** → Few unique values
Example: `gender`, `status` (repeated values)

In practice, cardinality is a key concept in **Indexing** and query optimization:

- Columns with **high cardinality** are good candidates for indexing because they help **filter rows efficiently**
- Columns with **low cardinality** are less effective for indexing (many rows match the same value)

Example:

```
SELECT COUNT(DISTINCT department) FROM employees;
```

This gives the cardinality of the `department` column.

Note (important distinction):

The term *cardinality* can also refer to **relationship types** between tables (1:1, 1:N, M:N). In this context, it describes how many records in one table relate to records in another.

Key takeaway:

Cardinality tells you **how unique the data is**, which directly impacts **query performance, indexing strategy, and data modeling**.

Q1. What do you mean by Data Inconsistency?

Ans. Data inconsistency occurs when the **same piece of data exists in multiple places but has different values**, leading to conflicts and unreliable results.

It typically arises in systems where data is **uplicated across tables, files, or systems** and updates are not applied uniformly.

Example:

- In one table, a customer's address = "Hyderabad"
 - In another table, the same customer's address = "Vizag"
- This mismatch is data inconsistency.

Common causes:

- **Data redundancy** (same data stored in multiple locations)
- **Lack of proper normalization**
- **Concurrent updates without proper controls**
- Application bugs or manual data entry errors

Problems it creates:

- Incorrect query results
- Loss of data reliability and trust
- Difficulties in reporting and decision-making

How to prevent it:

- Apply normalization (up to 3NF/BCNF where appropriate)
- Enforce constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE)
- Use transactions and ACID properties
- Avoid unnecessary duplication of data

Important point:

Data inconsistency breaks the **integrity of the database**, making the system unreliable for accurate information.

Q110. What do you mean by DELETE CASCADE constraint?

Ans. DELETE CASCADE (often written as **ON DELETE CASCADE**) is a **referential action on a foreign key** that automatically deletes related rows in a child table when the corresponding row in the parent table is deleted.

In other words: **delete the parent → all dependent child records are deleted automatically.**

Why it exists

It enforces **referential integrity** so you don't end up with orphan rows (child records pointing to a non-existent parent).

```
CREATE TABLE departments (  
  id INT PRIMARY KEY,  
  name TEXT  
);  
  
CREATE TABLE employees (  
  id INT PRIMARY KEY,  
  dept_id INT,  
  FOREIGN KEY (dept_id)  
    REFERENCES departments(id)  
    ON DELETE CASCADE  
);
```

Now:

```
DELETE FROM departments WHERE id = 10;
```

All **employees** with **dept_id = 10** are automatically deleted.

When to use

- True **ownership** relationships (e.g., order → order_items)
- Data that **should not exist independently** of the parent

Benefits

- Keeps data consistent without manual cleanup
- Simplifies application logic (no need to issue multiple deletes)

Cautions

- Can cause **large, cascading deletes** (performance impact)
- Risk of **accidental data loss** if used on the wrong relationship
- Often combined with transactions so you can **ROLLBACK** if needed

Contrast

- `ON DELETE RESTRICT / NO ACTION` → prevents deletion if children exist
- `ON DELETE SET NULL` → keeps child rows but nulls the foreign key

Key takeaway:

`ON DELETE CASCADE` automatically propagates deletes from parent to child tables, preserving referential integrity but requiring careful use.

Q111. What are the different flavors of SQL?

Ans. “Flavors” of SQL (often called **SQL dialects**) are **vendor-specific implementations** of the SQL standard. They all follow core SQL, but differ in **syntax, features, functions, and tooling**.

Here are the major ones you’ll encounter:

- **MySQL SQL (MySQL dialect)**
 - Widely used in web applications (LAMP stack)
 - Simple syntax, strong community support
 - Uses engines like InnoDB; features like `LIMIT`, `AUTO_INCREMENT`
- **PostgreSQL SQL**
 - Standards-compliant and feature-rich
 - Advanced capabilities: window functions, CTEs, JSONB, extensions
 - Strong for analytics and complex queries
- **Microsoft SQL Server (T-SQL)**
 - Uses **Transact-SQL (T-SQL)**
 - Rich procedural features (variables, loops, error handling)
 - Tight integration with the Microsoft ecosystem
- **Oracle Database (PL/SQL)**
 - Uses **PL/SQL** for procedural logic
 - Enterprise-grade features, high performance and scalability
 - Common in large corporate systems
- **SQLite SQL**
 - Lightweight, serverless, file-based
 - Limited feature set but very portable
 - Used in mobile apps, embedded systems

- **IBM Db2 SQL**
 - Enterprise-focused, used in mainframe and large-scale systems
 - Strong performance and analytics features
- **MariaDB SQL**
 - Fork of MySQL with additional features and optimizations
 - Open-source, drop-in replacement for MySQL in many cases

All flavors share **core SQL concepts** (**SELECT, JOIN, GROUP BY, etc.**), but each adds its own extensions—so moving between systems requires learning those **dialect-specific differences**.

Q112. Does an SQLite database support date time objects?

Ans. Yes—**SQLite does support date and time values**, but not as a separate dedicated data type like some other databases.

Instead, SQLite stores date/time in **three common formats**:

- **TEXT** → ISO format: `'YYYY-MM-DD HH:MM:SS'`
- **REAL** → Julian day number (fractional days since a fixed date)
- **INTEGER** → Unix timestamp (seconds since 1970-01-01)

SQLite provides **built-in date/time functions** to work with these values.

Built-in Date/Time Functions in SQLite

1. **DATE ()** → gives only date

```
SELECT DATE('2025-04-25 10:30:00');
```

👉 Output: `2025-04-25`

2. **TIME ()** → gives only time

```
SELECT TIME('2025-04-25 10:30:00');
```

👉 Output: `10:30:00`

3. **DATETIME()** → gives both

```
SELECT DATETIME('2025-04-25 10:30:00');
```

👉 Output: 2025-04-25 10:30:00

4. Current date/time

```
SELECT DATE('now');    -- today's date  
SELECT TIME('now');    -- current time  
SELECT DATETIME('now'); -- full current date & time
```

5. Add or subtract days (very useful)

```
SELECT DATE('now', '+5 days'); -- after 5 days  
SELECT DATE('now', '-2 days'); -- 2 days ago
```

👉 This is called **modifier**

6. **STRFTIME()** → custom format (advanced but useful)

```
SELECT STRFTIME('%Y', 'now'); -- year
```

👉 Output: 2026

Modifiers (Very Important)

SQLite allows **modifiers** to manipulate dates:

```
SELECT DATE('2025-04-25', '+5 days');  
SELECT DATETIME('now', '-1 hour');  
SELECT DATE('now', 'start of month');
```

SQLite does **not** have a dedicated DATE/TIME data type

`STRFTIME( )` is the most flexible function.

The format codes used inside `STRFTIME( )` in SQLite.



Date parts

- `%Y` → Year (4 digits) → 2026
- `%y` → Year (2 digits) → 26
- `%m` → Month (01–12)
- `%d` → Day (01–31)



Time parts

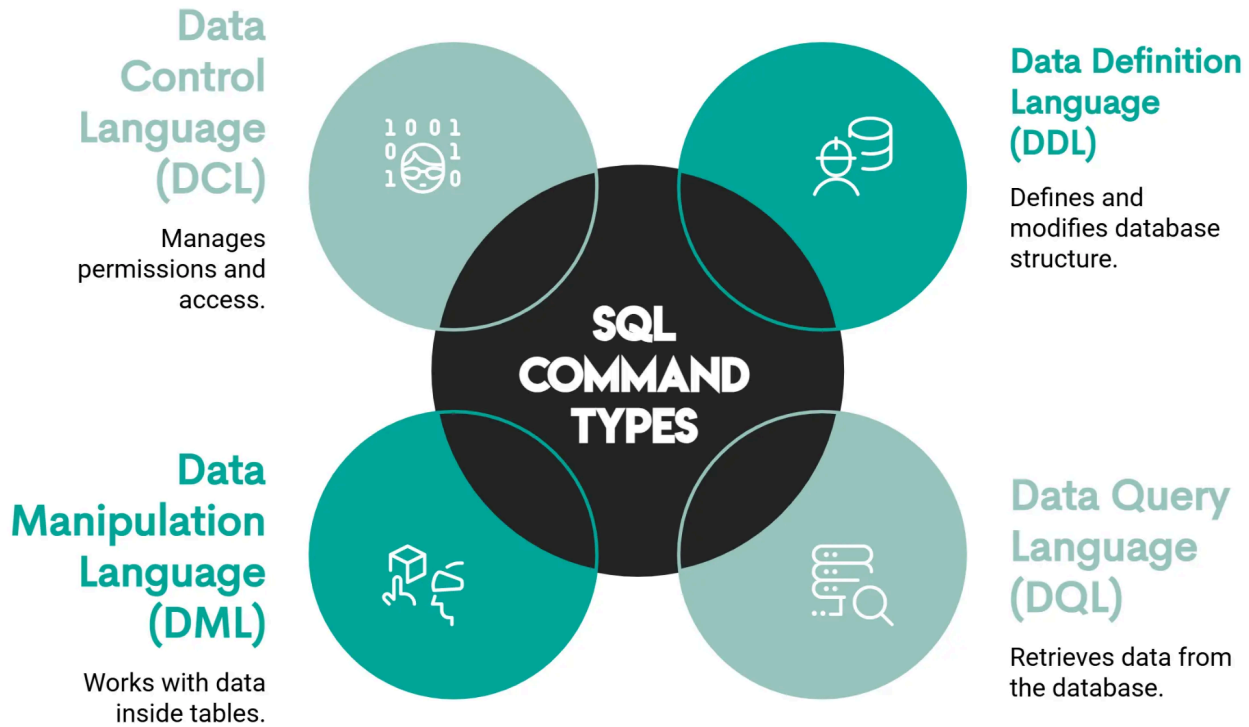
- `%H` → Hour (00–23)
- `%M` → Minutes (00–59)
- `%S` → Seconds (00–59)



Other useful ones

- `%w` → Day of week (0–6)
(0 = Sunday)
 - `%j` → Day of year (001–366)
 - `%W` → Week number
 - `%s` → Unix timestamp (seconds since 1970)
-

SQL SUBLANGUAGES



SQL COMMANDS COMPARISON

COMMAND	TYPE	EXPLANATION	EXAMPLE
 CREATE	DDL	Create a new object (table, view, index)	<pre>CREATE TABLE employees (id INT, name VARCHAR(50));</pre>
 ALTER	DDL	Modify an existing object's structure	<pre>ALTER TABLE employees ADD salary DECIMAL(10,2);</pre>
 DROP	DDL	Permanently remove an object and its data	<pre>DROP TABLE employees;</pre>
 RENAME	DDL	Change the name of an object	<pre>ALTER TABLE employees RENAME TO staff;</pre>
 TRUNCATE	DDL	Remove all rows from a table quickly	<pre>TRUNCATE TABLE employees;</pre>
 COMMENT	DDL	Add descriptive notes to database objects	<pre>COMMENT ON TABLE employees IS 'Stores employee records';</pre>
 SELECT	DQL	Query and return data from tables or views	<pre>SELECT name, salary FROM employees WHERE salary > 80000;</pre>
 INSERT	DML	Add new rows	<pre>INSERT INTO employees VALUES (1, 'Bob', 90000);</pre>
 UPDATE	DML	Change existing rows	<pre>UPDATE employees SET salary = 95000 WHERE id = 1;</pre>
 DELETE	DML	Remove specific rows	<pre>DELETE FROM employees WHERE id = 1;</pre>
 MERGE	DML	Insert or update rows based on matching conditions (upsert logic)	<pre>MERGE INTO employees e USING new employees n ON e.id = n.id WHEN MATCHED THEN UPDATE SET salary = n.salary WHEN NOT MATCHED THEN INSERT VALUES (n.id, n.name, n.salary);</pre>
 CALL	DML	Execute a stored procedure	<pre>CALL update_employee_salary(1, 95000);</pre>
 EXPLAIN PLAN	DML	Show how the database will execute a query	<pre>EXPLAIN SELECT * FROM employees WHERE salary > 80000;</pre>
 LOCK TABLE	DML	Control concurrent access by locking a table	<pre>LOCK TABLE employees IN EXCLUSIVE MODE;</pre>
 GRANT	DCL	Give privileges to users or roles	<pre>GRANT SELECT, INSERT ON employees TO analyst_role;</pre>
 REVOKE	DCL	Remove previously granted privileges	<pre>REVOKE INSERT ON employees FROM analyst_role;</pre>

SQLite does not support commands like TRUNCATE, MERGE, CALL, LOCK TABLE, GRANT, REVOKE, and COMMENT because it is designed as a lightweight, embedded database with limited advanced features. Instead of TRUNCATE, you use DELETE to remove all rows, and instead of MERGE, SQLite provides UPSERT using **ON CONFLICT**. Commands like CALL are not available because SQLite does not support stored procedures, and LOCK TABLE is unnecessary since SQLite manages locking automatically, while GRANT and REVOKE are not supported due to the absence of a user permission system. However, EXPLAIN (or EXPLAIN QUERY PLAN) is supported in SQLite and is used to show how a query will be executed internally, helping in understanding and optimizing query performance.

ROW_NUMBER() VS. RANK() VS. DENSE_RANK()

FEATURE	ROW_NUMBER()		RANK()		DENSE_RANK()	
Purpose	Assigns a unique sequential number to each row		Assigns a rank with gaps for ties		Assigns a rank without gaps for ties	
Handling Ties	No ties — each row gets a unique number		Ties get the same rank; the next rank is skipped		Ties get the same rank; the next rank is not skipped	
Ranking Sequence	Always increments by 1		Increments by number of tied rows		Always increments by 1	
Use Case	Deduplication, pagination		Ranking with awareness of position gaps		Ranking where consistent increments are needed	
Example for Ties	SCORE	ROW_NUMBER()	SCORE	RANK()	SCORE	DENSE_RANK()
	100	1	100	1	100	1
	100	2	100	1	100	1
	90	3	90	3	90	2